OTH REGENSBURG



Best Practices und Dokumentation

Dr. Timo Stadler

Programmierprojekt

OTH REGENSBURG

Agenda

1. Best Practices
    1. Warum Best Practices?
    2. Verschiedene Coding Guidelines
        1. MISRA C
        2. Stilrichtlinien
    3. Namenskonventionen und Formatierung
    4. Modularisierung und Header-Dateien
    5. Fehlerbehandlung
    6. Praxisbeispiel
2. Dokumentation
    1. Warum Doxygen?
    2. Wichtige Doxygen-Befehle
    3. Workflow in der Praxis

Best Practices

Warum Best Practices?

```
8   // Dear programmer:
9   // When I wrote this code, only god and
10  // I knew how it worked.
11  // Now, only god knows it!
12  //
13  // Therefore, if you are trying to optimize
14  // this routine and it fails (most surely),
15  // please increase this counter as a
16  // warning for the next person:
17  //
18  // total_hours_wasted_here = 254
19  //
20
```

Best Practices

Warum Best Practices?

- **Das Problem in C:**
  - **“C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off.” – Bjarne Stroustrup**
  - **C verzeiht keine Fehler:** Keines der üblichen Sicherheitsnetze (kein Garbage Collector, keine Boundary Checks)
  - **Code compiliert != Code ist korrekt:** Undefiniertes Verhalten (*Undefined Behavior*) führt zu späten, schwer auffindbaren Bugs
- **Direkter Nutzen für Entwickler:**
  - **Zeitersparnis bei der Fehlersuche:** Ein Großteil der Debugging-Zeit entfällt auf vermeidbare Routinefehler
  - **Zusammenarbeit:** Einheitlicher Stil verhindert Frust und Konflikte in Teamprojekten
  - **Wartbarkeit:** Code wird deutlich öfter gelesen als geschrieben (Lesbarkeit > Schreiben)

Best Practices

Verschiedene Coding Guidelines

- **Guidelines are a Spectrum**
  - Je nach Einsatzgebiet variieren Strenge und Fokus enorm

- Unterschied zwischen Standards und Richtlinien/Guidelines (C11, C17, Misra-C)
  - Standard beschreibt Syntax und Standardbibliotheken der Programmiersprache
  - Richtlinien, wie mit dem Standard umgegangen wird

- **Bekannte Entwicklungsrichtlinie in der Industrie:**
  - **MISRA C** (Motor Industry Software Reliability Association)
    - Sehr strenger Standard für höchste Funktionale Sicherheit
    - Verbot gefährlicher C-Features
      - **Kein `goto`**
      - **Keine Standard C-Datentypen (`int, long, short`)**
      - **Keine dynamische Speicherverwaltung (`malloc(), free()`)**
      - **Uvm**…

  - **Unternehmensspezifische Styleguides (z. B. Google, Linux Kernel)**
    - Klare Vorgaben für Benennung, Einrückung und Dateistruktur

Best Practices

Verschiedene Coding Guidelines

- **Ihr Programmierprojekt:**
  - **Formatierung: LLVM Coding Standard**
  - **C: Linux kernel coding style**
  - **C++: Google C++ Style Guide**

- **In der Praxis müssen Sie diese Standards nicht auswendig im Kopf haben**
- **Dafür gibt es Tools**
  - **Formatter:** *clangd* für automatisches Formatieren des Codes
  - **Statische Analyse:** *Clang-Tidy* zur Fehlersuche im Code
  - **Sanitizer:** *AddressSanitizer* zum finden von Speicherzugriffsfehlern

Best Practices

Verschiedene Coding Guidelines

- **Was Sie generell immer im Kopf behalten sollten:**

1. **Immer Warnungen aktivieren & ernst nehmen:**
   - *Immer* mit -Wall -Wextra –Werror kompilieren. **Eine Warnung ist ein Fehler!**
2. **Sichere String-Funktionen:**
   - Verbot von `strcpy` und `sprintf` – konsequente Nutzung von `snprintf`
3. **Zeiger-Hygiene:**
   - Jeden Zeiger sofort bei Deklaration auf `NULL` setzen und vor dem Zugriff prüfen.
4. **Speicher-Symmetrie:**
   - Zu jedem `malloc()` gehört direkt im Entwurf ein `free()`
5. **Strikte Dateitrennung:**
   - Keine globalen Variablen (`extern`), sauber getrennte .c- und .h-Dateien mit *Include Guards*.
6. **Sprechende Namen:**
   - Keine kryptischen Abkürzungen (besser `counter` statt `cnt`).

Best Practices

Namenskonventionen und Formatierung (Linux Kernel Guide)

- **Variablen & Funktionen:** Immer in *snake_case*
  - *Beispiel:* `packet_length, read_temperature_sensor()`
- **Konstanten & Makros:** Immer in *UPPER_CASE*
  - *Beispiel:* `#define MAX_BUFFER_SIZE 1024, const int TIMEOUT_MS = 500;`
- **Einrückung:**
  - Einheitlich (4 Leerzeichen statt Tabs zur Vermeidung von Anzeigefehler)
- **Klammersetzung:** Die öffnende geschweifte Klammer { steht direkt in derselben Zeile
  - Beispiel:

```
if (status == SUCCESS) {
    process_data();
}
```

Best Practices

Namenskonventionen und Formatierung (Linux Kernel Guide)

```
int calcXYZ(int a,int b){
if(a>b){return a-b;}else{
      return a+b;
}
}
int main(){
  int x=5;int Y=10;
    int ret_Val_abc=calcXYZ(x,Y);
  printf("Result: %d\n",ret_Val_abc);
  return 0;
}
```



- Chaos bei den Namenskonventionen
- Schlechte Formatierung und Einrückung
- Fehlende Lesbarkeit

Best Practices

Namenskonventionen und Formatierung (Linux Kernel Guide)

```c
static int compute_sum_or_difference(int a, int b)
{
    if (a > b) {
        return a - b;
    } else {
        return a + b;
    }
}

int main(void)
{
    int x = 5;
    int y = 10;
    int result;

    result = compute_sum_or_difference(x, y);

    printf("Result: %d\n", result);

    return 0;
}
```

- Klammersetzung bei Funktionen (The "Next Line" Rule)
- Leerzeichen statt Tabs
- Deklarationen am Blockanfang
- *Static* von Hilfsfunktionen

Best Practices

Modularisierung und Header-Dateien

- **Grundprinzip: Trennung**
- **Deklaration vs. Definition:** Code wird logisch in Module aufgeteilt, um Übersichtlichkeit und Kompilierungsgeschwindigkeit zu maximieren.

- **Header-Dateien *(.h)*:**
  - Dienen als **Schnittstelle** (API) nach außen.
  - Enthalten Funktionsprototypen, `struct`-Definitionen, `typedef`-Anweisungen und *globale Konstanten/Makros*.
- **C-Quelldateien *(.c)*:**
  - Dienen als **Implementierung**.
  - Enthalten den tatsächlichen Programmtext (Funktionsrümpfe) und lokale Hilfsfunktionen (`static`).

Best Practices

Modularisierung und Header-Dateien

- **Best Practices:**
  - **Selbstbeinhaltende Header:** Jeder Header muss alle anderen Header, die er für seine eigenen Typen benötigt, selbst einbinden (keine impliziten Abhängigkeiten).
  - **Möglichst keine globalen Variablen (extern):** Modulzustände sollten nicht über globale Variablen geteilt werden.
  - **Sichtbarkeit einschränken:** Hilfsfunktionen, die nur innerhalb einer .c-Datei gebraucht werden, mit dem Schlüsselwort static versehen.
- **Include Guards** (Schutz vor mehrfachem Einbinden)
  - **Problem:**
    - Wenn ein Header mehrfach in eine Übersetzungseinheit eingebunden wird, führt das zu Syntaxfehlern (Mehrfachdefinitionen).
  - **Lösung**:

counter.h

```c
#ifndef COUNTER_H
#define COUNTER_H

/**
 * @brief Erhöht den Zähler um 1.
 */
void counter_increment(void);


#endif // COUNTER_H
```

Best Practices

Fehlerbehandlung

- **Das Grundproblem in C**
  - **Keine Exceptions:**
    - C besitzt im Gegensatz zu C++, Java oder Python kein try-catch-Ausnahmehandling.
  - **Explizite Handhabung notwendig:**
    - Fehler explizit abfangen und prüfen, da C-Programme andernfalls bei Fehlern unbemerkt mit falschen Daten weiterrechnen oder abstürzen (*Segmentation Fault*).
- **Gängige Muster zur Fehlererkennung**
  - **Rückgabewerte als Status:** Funktionen geben oft einen Fehlercode zurück
    - (`return 0`; für Erfolg, -1 oder einen speziellen Enum-Wert für Fehler).
  - **Zeiger als Indikator:**
    - Funktionen, die fehlschlagen können, geben bei Misserfolg NULL zurück (z. B. `malloc()`).
  - **Out-Parameter für Daten, Rückgabewert für Status:**
    - Der eigentliche Status (Erfolg/Fehler) wird als bool oder int zurückgegeben.
    - Das eigentliche Ergebnis wird über einen Zeiger-Parameter übergeben.
    - *Beispiel:* `bool sensor_read(float *out_value);`
- **Denken Sie immer an mögliche Fehler und räumen Sie ihren Code auf!**

Best Practices

Fehlerbehandlung

- Beispiel

```c
int process_user_data(const char *filename) {

    // 1. Fehler von fopen wird ignoriert! Wenn Datei fehlt -> Absturz.
    FILE *file = fopen(filename, "r");

    // 2. Fehler von malloc wird ignoriert! Wenn kein Speicher da ist -> Absturz.
    char *buffer = malloc(1024);

    // 3. Wenn file oder buffer NULL sind, knallt es hier unweigerlich.
    fread(buffer, 1, 1024, file);

    // ... verarbeite Daten ...

    fclose(file);
    free(buffer);

    return 0;
}
```



Best Practices

Fehlerbehandlung

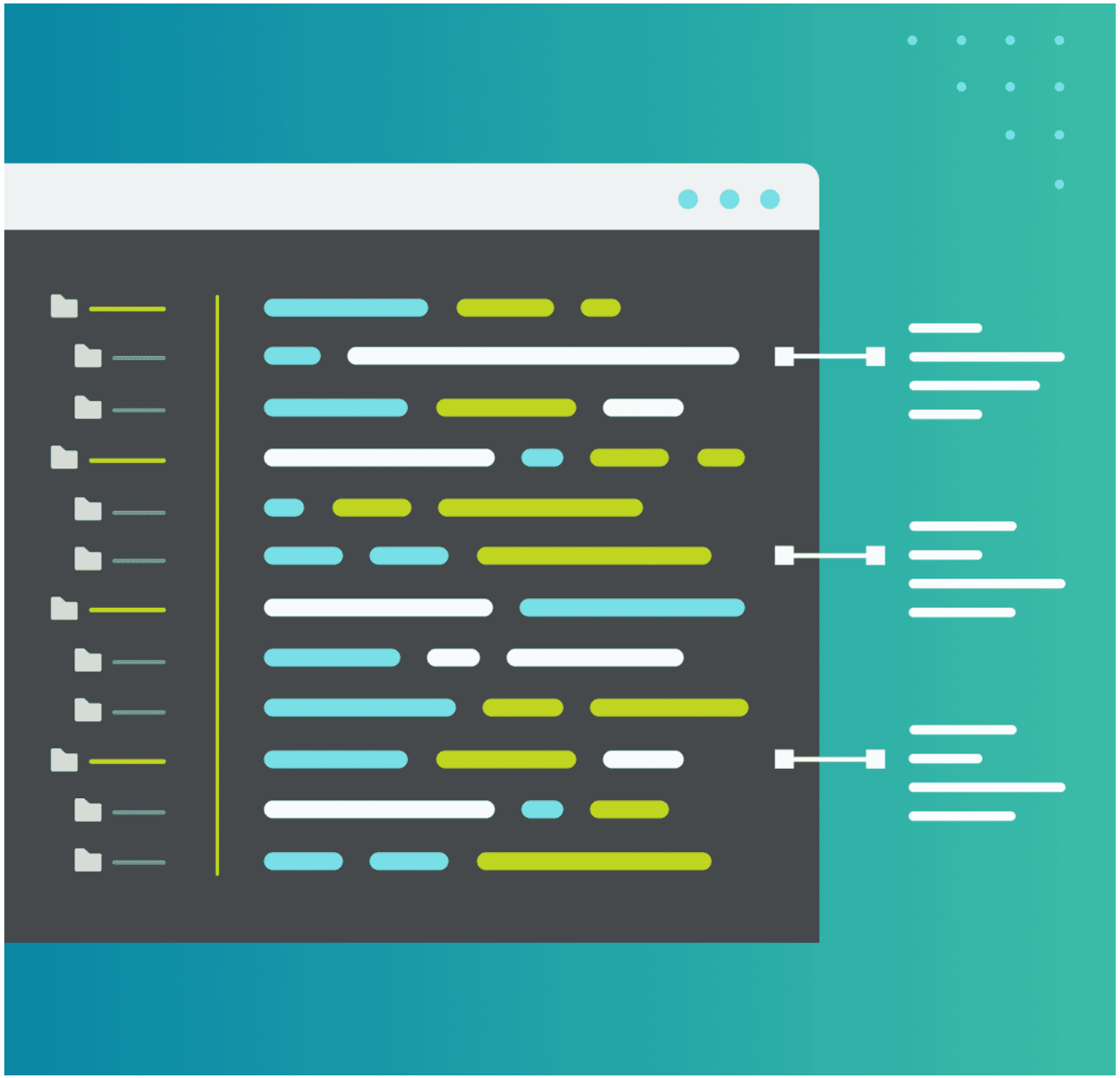
```c
// 1. Eindeutige Status- und Fehlercodes als Enum definieren
typedef enum {
    STATUS_OK = 0,
    ERROR_INVALID_PARAM,
    ERROR_FILE_OPEN,
    ERROR_MEM_ALLOC,
    ERROR_READ_FAILED
} ResultCode;
```

```c
ResultCode process_user_data(const char *filename) {
    if (filename == NULL) {
        return ERROR_INVALID_PARAM;
    }

    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        return ERROR_FILE_OPEN;
    }

    char *buffer = malloc(1024);
    if (buffer == NULL) {
        // WICHTIG: Vor dem Fehler-Exit die geöffnete Datei schließen!
        fclose(file);
        return ERROR_MEM_ALLOC;
    }

    size_t bytes_read = fread(buffer, 1, 1024, file);
    if (bytes_read == 0 && ferror(file)) {
        fclose(file);
        free(buffer);
        return ERROR_READ_FAILED;
    }
```



Dokumentation

Dokumentation

Wie wird in Doxygen kommentiert?

- Damit Doxygen einen Kommentar als Dokumentation erkennt, muss er ein **spezielles Einleitungszeichen** besitzen:
  - **Variante 1 (Dreifach-Slash):**
    - /// oder //!
  - **Variante 2 (Javadoc-Stil mit Sternchen):**
    - /*! */ oder **/** ... */ -> Präferiert**
  - Der Doxygen-Kommentar muss **direkt vor** dem zu dokumentierenden Element (Funktion, Struct) stehen.
- **Was, wenn ich *nicht* will, dass etwas in der Doku landet?**
  - z.B. interne Implementierungsdetails oder private Hilfsvariablen
  - **Normale Kommentare nutzen!**
- **Wie baue ich die Dokumentation?**
  - Im Root des Projekts einmalig eine Standard-Konfigurationsdatei (Doxyfile) erstellen mit: ***doxygen -g***
  - Bei jeder Änderung die Dokumentation bauen mit: ***doxygen***
- **Wo finde ich das Ergebnis?**
  - Standardmäßig erstellt Doxygen in deinem Projektverzeichnis zwei neue Ordner:
    - **html/** – Web-Dokumentation und **latex/** für PDF generierung
    - **html/ Ordner** zur ***.gitignore*** hinzufügen da generierte Files

counter.h

```c
#ifndef COUNTER_H
#define COUNTER_H

/**
 * @brief Erhöht den Zähler um 1.
 */
void counter_increment(void);

/**
 * @brief Berechnet die Summe zweier Werte.
 *
 * @param[in] a Erster Summanden-Wert.
 * @param[in] b Zweiter Summanden-Wert.
 * @return Die berechnete Summe.
 */
int calculate_sum(int a, int b);

#endif // COUNTER_H
```

Dokumentation

Workflow in der Praxis

- https://www.doxygen.nl/
- Doxygen hat das Wiki seines Projekts in Doxygen gemacht:
  - https://www.doxygen.nl/manual/index.html
- Vielzahl von Befehlen (Tags), die mit einem @-**Zeichen** eingeleitet werden.
  - **Allgemeine Beschreibung**
    - ***@brief <Text>:*** kurze, prägnante Ein-Zeilen-Zusammenfassung (erscheint in Tooltips).
    - ***@details <Text>:*** ausführliche Beschreibung, die nach dem Brief-Text folgt.
    - ***@file <Dateiname>:*** Dokumentiert eine gesamte Quelldatei (steht meist ganz oben in einer .h- oder .c-Datei).
  - **Funktionen & Parameter**
    - ***@param[in]*** **<Name> <Beschreibung>**: Beschreibt einen Übergabeparameter. Mit dem Zusatz in Klammern ([in], [out] oder [in,out]) lässt sich die Richtung des Datenflusses klar kennzeichnen.
    - ***@return*** **<Beschreibung>**: Beschreibt den Rückgabewert einer Funktion.
    - ***@retval*** **<Wert> <Beschreibung>**: Dokumentiert spezifische Rückgabewerte (z. B. bei Fehlercodes wie 0 für Erfolg und -1 für Fehler).
- **Es gibt noch viele mehr, diese wenigen reichen aber für 99% der Dokumentation aus!**